

MAWLANA BHASHANI SCIENCE AND TECHNOLOGY UNIVERSITY

SANTOSH, TANGAIL-1902



DEPARTMENT OF INFORMATION AND COMMUNICATION TECHNOLOGY

Lab Report

Lab Report No : 08

Lab Report on : Working with JSON and Third-Party APIs Using JavaScript: Implementation
and Postman Testing

Course Title : Web Application Development Lab

Course Code : ICT-3204

Submitted By	Submitted To
Name: Kuldip Saha Mugdha ID: IT-22018 3 rd Year, 2 nd Semester Session: 2021-2022 Dept. of ICT, MBSTU	Mr. Md. Anowar Kabir Lecturer DEPARTMENT OF INFORMATION AND COMMUNICATION TECHNOLOGY MAWLANA BHASHANI SCIENCE AND TECHNOLOGY UNIVERSITY

Date of Performance: 12/01/26

Date of Submission: 18/01/26

Introduction:

Modern web applications frequently interact with external services through Application Programming Interfaces (APIs). APIs allow applications to exchange data in structured formats such as JSON, enabling dynamic content generation and validation services. In this lab, JavaScript is used to access and process JSON data, test third-party APIs using Postman, and develop a simple web interface that provides user feedback based on API responses. This lab demonstrates practical skills in API testing, JSON handling, and client-side web development.

Objectives:

The main objectives of this lab are:

1. To understand the structure and usage of JSON files
2. To access and manipulate JSON data using JavaScript
3. To install and use Postman for API testing
4. To test Email Reputation and Weather APIs
5. To display API responses in JSON format using JavaScript
6. To develop a web page that provides feedback based on user input and API responses.

Tools and Technologies Used:

- HTML5
- CSS3
- JavaScript (ES6)
- JSON
- Postman API Testing Tool
- Email Reputation API (Abstract API)
- Weather API
- Web Browser (Google Chrome)

Procedures:

Step 1: Creating and Accessing a JSON File

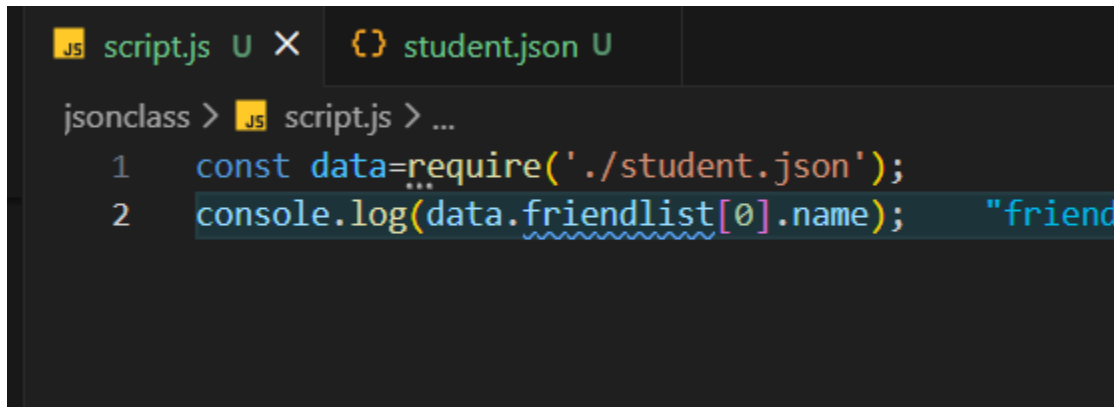
A meaningful JSON file was created containing student information. JavaScript was used to read and display the JSON data in the browser console.

Source Code:

Sample JSON File (student.json)

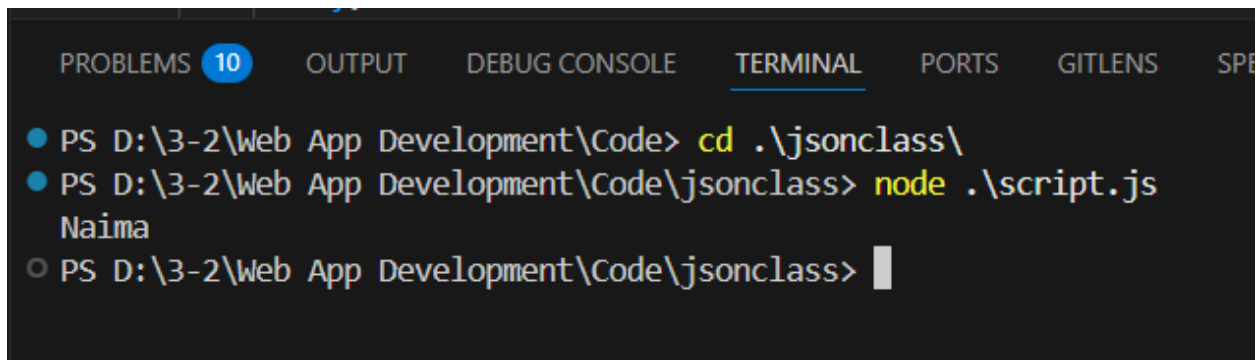
```
...  script.js U  student.json U X
jsonclass > {} student.json > {} course > {} 1
1  {
2      "id": "IT22018",
3      "name": "Kuldip Saha Mugdha",    "Kuldip": Unknown word.
4      "department": "Information and Communication Technology",
5      "age": 21,
6      "course": [
7          {
8              "courseID": "ICT3203",
9              "courseName": "Web Application Development",
10             "credits": 3
11         },
12         [
13             {
14                 "courseID": "ICT3204",
15                 "courseName": "Web Application Development Lab",
16                 "credits": 1
17             }
18         ],
19         "friendlist": [    "friendlist": Unknown word.
20             {
21                 "id": "IT22001",
22                 "name": "Naima",    "Naima": Unknown word.
23                 "dist": "Gazipur"    "Gazipur": Unknown word.
24             },
25             {
26                 "id": "IT22025",
27                 "name": "Sagor",    "Sagor": Unknown word.
28                 "dist": "Dhaka"
29             },
30             {
31                 "id": "IT22032",
32                 "name": "Shourav",    "Shourav": Unknown word.
33                 "dist": "Comilla"    "Comilla": Unknown word.
34             }
35         ]
36     }
```

JavaScript to Access JSON File (script.js)



```
script.js U X student.json U
jsonclass > script.js > ...
1 const data=require('./student.json');
2 console.log(data.friendlist[0].name); "friend"
```

Output:



```
PROBLEMS 10 OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS SPE
• PS D:\3-2\Web App Development\Code> cd .\jsonclass\
• PS D:\3-2\Web App Development\Code\jsonclass> node .\script.js
Naima
○ PS D:\3-2\Web App Development\Code\jsonclass> 
```

Step 2: Installing Postman

Postman software was downloaded from the official website and installed. It was used to test API endpoints by sending HTTP GET requests and analyzing JSON responses.

Step 3: Testing Email Reputation API

The Email Reputation API was tested in Postman by passing an email address and API key as query parameters. The API response was checked to determine whether the email was deliverable.

Source Code:

JavaScript Code to Test Email API

```
code.js U X
API_with_json > code.js > ...
1
2 const url='https://emailreputation.abstractapi.com/v1/';
3 const api_key='a64f2ab1d01e49b0b64ee78ad84beeef';
4 const email='mugdha.saha67@gmail.com';
5
6 fetch(`${url}?api_key=${api_key}&email=${email}`)
7   .then(response => response.json())
8   .then(data => {
9     if(data.email_deliverability.status==='deliverable'){
10       console.log("The email address is valid and deliverable.");
11     } else{
12       console.log("The email address is not deliverable.");
13     }
14   })
```

Postman Response:

```
Body Cookies Headers (12) Test Results (1/1) 200 OK • 1.05 s •
{} JSON Preview Visualize
1 {
2   "email_address": "mugdha.saha67@gmail.com",
3   "email_deliverability": {
4     "status": "deliverable",
5     "status_detail": "valid_email",
6     "is_format_valid": true,
7     "is_smtp_valid": true,
8     "is_mx_valid": true,
9     "mx_records": [
10     "alt1.gmail-smtp-in.l.google.com",
11     "alt2.gmail-smtp-in.l.google.com",
12     "gmail-smtp-in.l.google.com",
13     "alt4.gmail-smtp-in.l.google.com",
```

Step 4: Testing Weather API

The OpenWeatherMap API was tested in Postman using a city name and API key. Weather information such as temperature and condition was returned in JSON format.

Source code:

JavaScript Code to Test Weather API

```
JS weather_api_test.js U <> index.html ...webpage_for_email_weather U # style.css ...webpage_for_email_weather U JS sc
JS > web-lab-8 > JS weather_api_test.js > [🔗] apiKey
1  const url = 'https://api.openweathermap.org/data/2.5/weather';
2  const apiKey = '2d932e0c34e29da5988a0942c74be895'; // replace with your key
3  const city = 'Dhaka';
4
5  fetch(`${url}?q=${city}&appid=${apiKey}&units=metric`)
6    .then(response => response.json())
7  > .then(data => {
8  >   if (data.cod === 200) {
9  >     console.log('Weather API is working ✅');
10 >     console.log('City:', data.name);
11 >     console.log('Temperature:', data.main.temp + '°C');
12 >     console.log('Condition:', data.weather[0].description);
13 >
14 >     // full JSON output
15 >     console.log('Full JSON:', data);
16 >   } else {
17 >     console.log('Weather API error ❌');
18 >     console.log(data.message);
19 >   }
20 > })
21 > .catch(error => {
22 >   console.error('Fetch error:', error);
23 > });
```

Postman response:

The image shows the Postman interface for a GET request to the OpenWeatherMap API. The request URL is `https://api.weatherapi.com/v1/current.json?key=81e3ff69832a4003843143111261601&q=Dhaka&aqi...`. The request is configured with parameters: `key` (81e3ff69832a4003843143111261601), `q` (Dhaka), and `aqi` (yes). The response is a 200 OK status with a response time of 235 ms and a size of 1.06 KB. The response body is a JSON object representing weather data for Dhaka, Bangladesh.

Key	Value	Description
key	81e3ff69832a4003843143111261601	
q	Dhaka	
aqi	yes	
Key	Value	Description

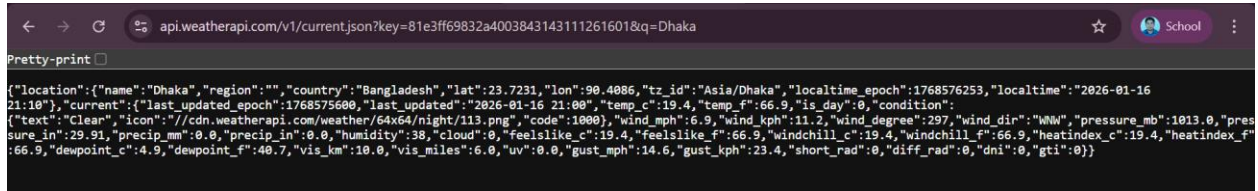
Body Cookies Headers (23) Test Results (1/1) 200 OK • 235 ms • 1.06 KB Save Response

JSON Preview Visualize

```
1  {
2    "location": {
3      "name": "Dhaka",
4      "region": "",
5      "country": "Bangladesh",
6      "lat": 23.7231,
7      "lon": 90.4086,
8      "tz_id": "Asia/Dhaka",
9      "localtime_epoch": 1768574683,
10     "localtime": "2026-01-16 20:44"
11   },
12   "current": {
```

In browser:

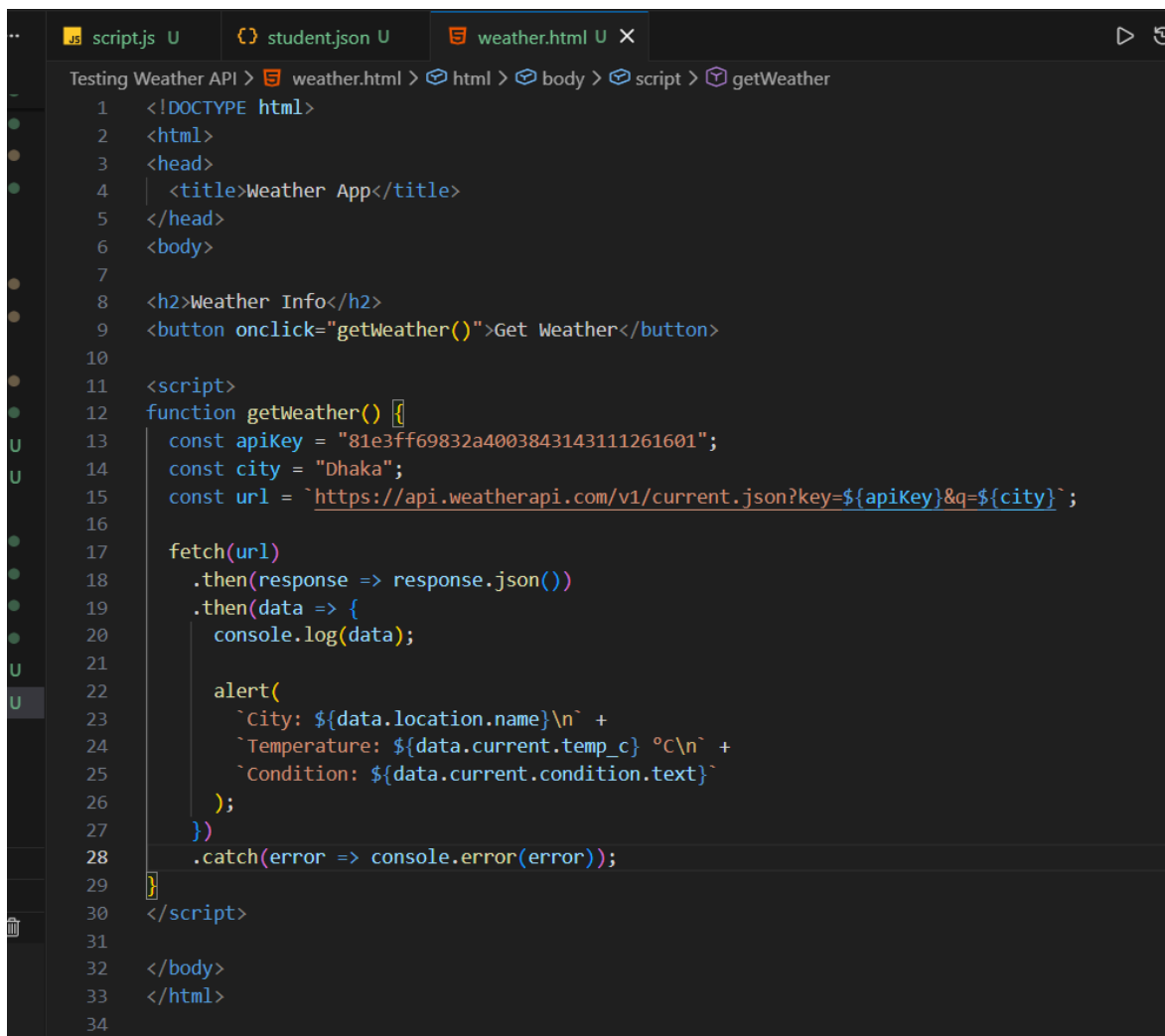
<https://api.weatherapi.com/v1/current.json?key=81e3ff69832a4003843143111261601&q=Dhaka>



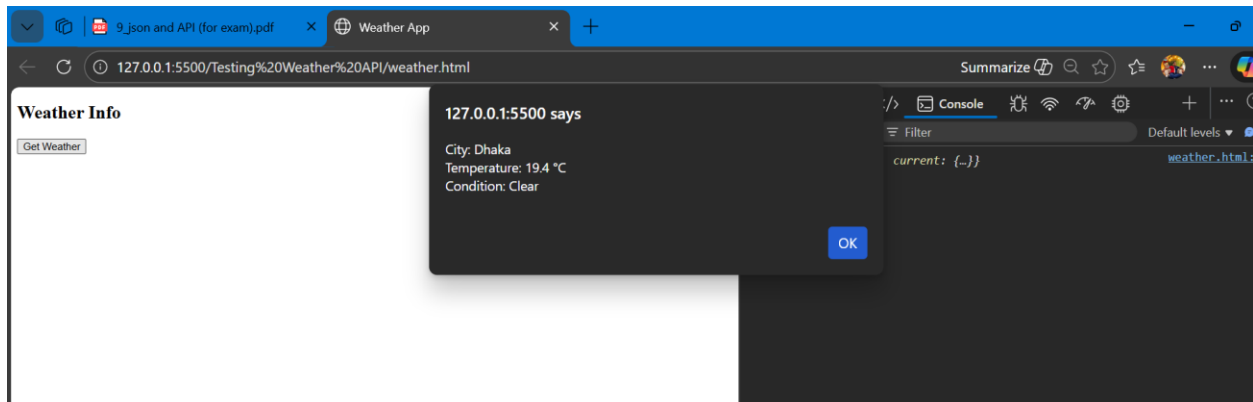
Step 5: Displaying API JSON Data Using JavaScript

JavaScript `fetch()` was used to retrieve API responses and display full JSON data in the browser console.

Source Code:



Output:



Step 6: Developing a Web Page for Email and Weather Feedback

A web page was developed where users can:

- Enter an email address to check deliverability
- Enter a city name to view weather information

The page provides separate feedback for each input based on API responses.

Source Code

```
isting Weather API > feedback.html > ...
1  <!DOCTYPE html>
2  <html>
3  <head>
4    <title>Weather Feedback App</title>
5
6    <style>
7      body {
8        font-family: 'Segoe UI', Tahoma, sans-serif;
9        background: linear-gradient(135deg, #74ebd5, #9face6);
10       min-height: 100vh;
11       display: flex;
12       justify-content: center;
13       align-items: center;
14     }
15
16     .container {
17       background: white;
18       padding: 25px 30px;
19       width: 380px;
20       border-radius: 15px;
21       box-shadow: 0 10px 25px rgba(0,0,0,0.2);
22     }
23
24     h2 {
25       text-align: center;
26       margin-bottom: 15px;
27       color: #333;
28     }
29
30     input {
31       width: 100%;
32       padding: 10px;
33       border-radius: 8px;
34       border: 1px solid #ccc;
35       margin-bottom: 10px;
```



```

button {
  width: 100%;
  padding: 10px;
  background: #4facfe;
  color: white;
  border: none;
  border-radius: 8px;
  font-size: 15px;
  cursor: pointer;
  transition: 0.3s;
}

button:hover {
  background: #00c6ff;
}

.feedback {
  margin-top: 12px;
  padding: 10px;
  border-radius: 8px;
  font-size: 14px;
  line-height: 1.6;
}

.success {
  background: #e6fffa;
  color: #065f46;
}

.error {
  background: #fee2e2;
  color: #7f1d1d;
}

```

```

  hr {
    margin: 20px 0;
  }
</style>
</head>

<body>

<div class="container">

  <h2>✉ Email Feedback</h2>
  <input type="text" id="email" placeholder="Enter your email">
  <button onclick="checkEmail()">Submit Email</button>
  <div id="emailFeedback" class="feedback"></div>

  <hr>

  <h2>☀ City Weather</h2>
  <input type="text" id="city" placeholder="Enter city name">
  <button onclick="getWeather()">Check Weather</button>
  <div id="weatherFeedback" class="feedback"></div>

</div>

<script>
function checkEmail() {
  const email = document.getElementById("email").value;
  const feedback = document.getElementById("emailFeedback");
  if (email.length > 0) {

```

```

if (email === "") {
  feedback.className = "feedback error";
  feedback.textContent = "✖ Email cannot be empty.";
} else if (email.includes("@") && email.includes(".")) {
  feedback.className = "feedback success";
  feedback.textContent = "✅ Email looks valid!";
} else {
  feedback.className = "feedback error";
  feedback.textContent = "⚠ Invalid email format.";
}
}

function getWeather() {
  const city = document.getElementById("city").value;
  const feedback = document.getElementById("weatherFeedback");

  if (city === "") {
    feedback.className = "feedback error";
    feedback.textContent = "✖ Please enter a city name.";
    return;
  }

  const apiKey = "81e3ff69832a4003843143111261601";
  const url = `https://api.weatherapi.com/v1/current.json?key=${apiKey}&q=${city}`;

  feedback.className = "feedback";
  feedback.textContent = "🌤 Fetching weather...";

  fetch(url)

```

```

    fetch(url)
      .then(res => res.json())
      .then(data => {
        if (data.error) {
          feedback.className = "feedback error";
          feedback.textContent = "✖ " + data.error.message;
          return;
        }

        const temp = data.current.temp_c;
        const condition = data.current.condition.text;

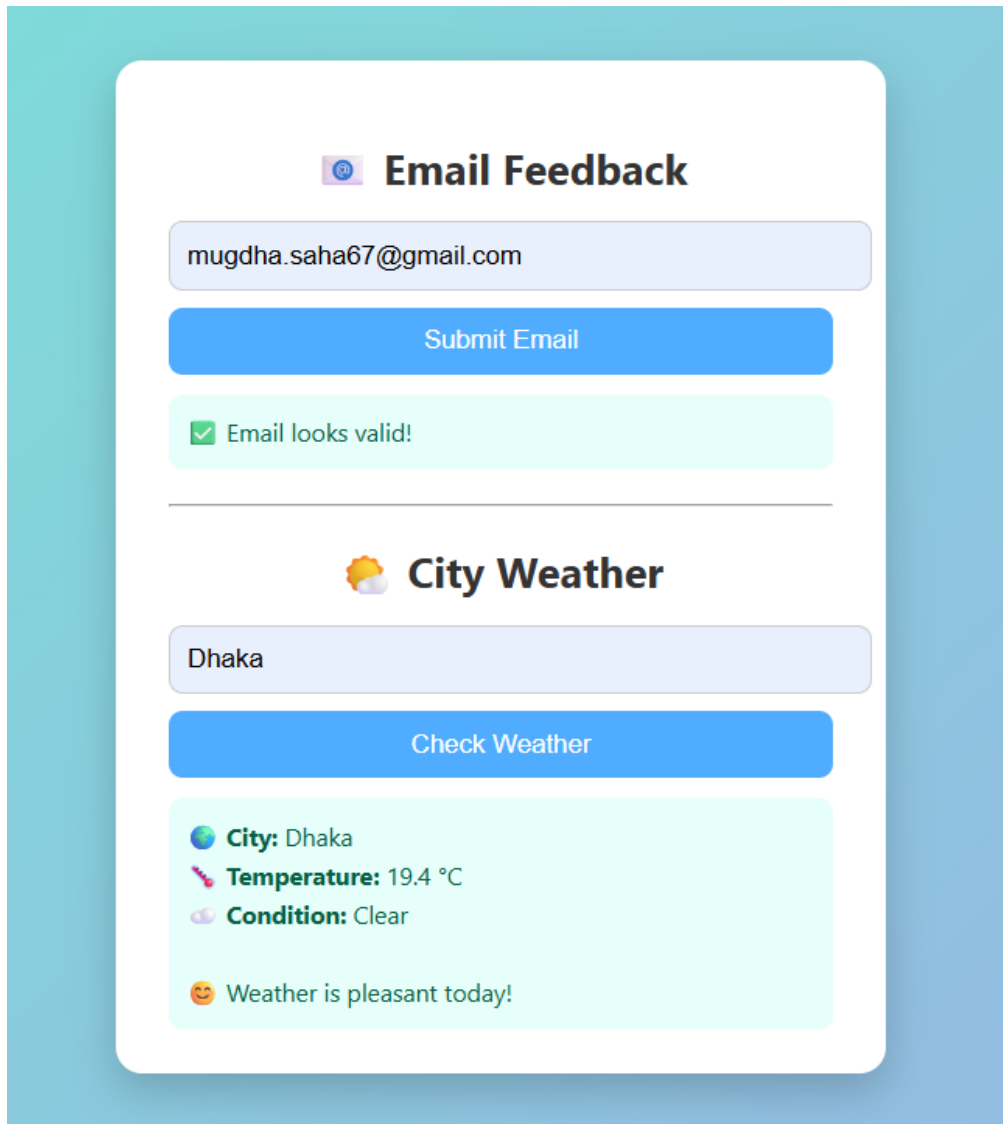
        let mood;
        if (temp > 30) mood = "🔥 It's hot! Stay hydrated.";
        else if (temp < 15) mood = "❄ It's cold! Wear warm clothes.";
        else mood = "😊 Weather is pleasant today!";

        feedback.className = "feedback success";
        feedback.innerHTML = `
          🌍 <b>City:</b> ${data.location.name}<br>
          🌡 <b>Temperature:</b> ${temp} °C<br>
          ☁ <b>Condition:</b> ${condition}<br><br>
          ${mood}
        `;
      })
      .catch(() => {
        feedback.className = "feedback error";
        feedback.textContent = "✖ Network or API error.";
      });
  }
</script>

</body>

```

6. Output Screenshots:



The screenshot displays a web interface with two main sections: 'Email Feedback' and 'City Weather'. The 'Email Feedback' section includes a text input field containing 'mugdha.saha67@gmail.com', a blue 'Submit Email' button, and a green feedback message 'Email looks valid!'. The 'City Weather' section features a text input field with 'Dhaka', a blue 'Check Weather' button, and a green box displaying weather details for Dhaka: 'City: Dhaka', 'Temperature: 19.4 °C', 'Condition: Clear', and a smiley face with the text 'Weather is pleasant today!'.

Email Feedback

mugdha.saha67@gmail.com

Submit Email

✓ Email looks valid!

City Weather

Dhaka

Check Weather

🌍 **City:** Dhaka
🌡️ **Temperature:** 19.4 °C
☁️ **Condition:** Clear

😊 Weather is pleasant today!

Conclusion:

In this lab, JSON data handling, API testing, and web-based user interaction were successfully implemented using JavaScript and Postman. The Email Reputation and Weather APIs were tested and integrated into a web interface that provides real-time feedback based on user input. This lab enhanced practical understanding of REST APIs, JSON structures, and client-side development, which are essential skills for modern web and software engineering applications.